Practical: Entity Framework Core, CRUD, Authentication and Authorization

Practical 1 — Product Table: Insert Operation using Entity Framework Core

Objective: Create a Product table in SQL Server and perform the Insert operation using Entity Framework Core.

Step 1 — Create ASP.NET Core MVC Project

Create an ASP.NET Core Web App (Model-View-Controller) in Visual Studio. Use the project name EFProductDemo.

Step 2 — Install Entity Framework Core Packages

Open Tools → NuGet Package Manager → Package Manager Console and run:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
```

Step 3 — Create Product Model

Create Models/Product.cs:

```
using System.ComponentModel.DataAnnotations;

namespace EFProductDemo.Models
{
    public class Product
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }
        public decimal Price { get; set; }
        public int Quantity { get; set; }
    }
}
```

Step 4 — Create Database Context

Create Data/ApplicationDbContext.cs:

```
using EFProductDemo.Models;
using Microsoft.EntityFrameworkCore;

namespace EFProductDemo.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }
        public DbSet<Product> Products { get; set; }
    }
}
```

ApplicationDbContext manages the database connection and provides Entity Framework Core with the configuration required to communicate with the SQL Server database. options carries the database configuration from Program.cs to ApplicationDbContext.

DbSet<Product> Products represents the **Products table** and allows the application to

Step 5 — Add Connection String

Open appsettings.json and add:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
```

```
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=EFProductDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

Step 6 — Configure Entity Framework Core

Open Program.cs:

```
using EFProductDemo.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseSqlServer(
            builder.Configuration.GetConnectionString(
                "DefaultConnection"
            )
        )
    );

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Product}/{action=Index}/{id?}"
);

app.Run();
```

Program.cs configures the ASP.NET Core MVC application by registering MVC services, connecting ApplicationDbContext to SQL Server using the DefaultConnection from appsettings.json, and building the application.

It configures error handling, HTTPS, static files, routing, and authorization, defines Product/Index as the default route, and finally starts the application.

Step 7 — Create Database using Migration

In Package Manager Console run:

```
Add-Migration InitialCreate
Update-Database
```

Entity Framework creates the EFProductDB database and Products table.

Step 8 — Create Product Controller

Create Controllers/ProductController.cs:

```
using EFProductDemo.Data;
using EFProductDemo.Models;
using Microsoft.AspNetCore.Mvc;

namespace EFProductDemo.Controllers
{
    public class ProductController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductController(
            ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var products = _context.Products.ToList();

            return View(products);
        }

        [HttpPost]
        public IActionResult Insert(Product product)
        {
            if (ModelState.IsValid)
            {
                _context.Products.Add(product);
                _context.SaveChanges();
                return RedirectToAction("Index");
            }

            return View("Index", _context.Products.ToList());
        }
    }
}
```

Step 9 — Create Product View

Create Views/Product/Index.cshtml:

```
@model List<EFProductDemo.Models.Product>

@{
    ViewData["Title"] = "Product";
}

<div class="container">

    <h1>Product Management</h1>

    <div class="box">

        <h2>Insert Product</h2>

        <form method="post"
              action="/Product/Insert">
```

```
            <label>Product Name</label>

            <input type="text"
                   name="Name"
                   required />

            <label>Price</label>

            <input type="number"
                   name="Price"
                   step="0.01"
                   required />

            <label>Quantity</label>

            <input type="number"
                   name="Quantity"
                   required />

            <button type="submit">
                Insert Product
            </button>

        </form>

    </div>

    <div class="box">

        <h2>Products</h2>

        <table>

            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Price</th>
                <th>Quantity</th>
            </tr>

            @foreach (var product in Model)
            {
                <tr>
                    <td>@product.Id</td>
                    <td>@product.Name</td>
                    <td> @product.Price</td>
                    <td>@product.Quantity</td>
                </tr>
            }

        </table>

    </div>

</div>

<style>

    body {
        font-family: Arial;
        background: #f5f7fa;
    }
```

```
    .container {
        max-width: 800px;
        margin: 30px auto;
    }

    .box {
        background: white;
        padding: 25px;
        margin-bottom: 25px;
        border-radius: 10px;
        box-shadow: 0 3px 10px #ccc;
    }

    input {
        display: block;
        width: 95%;
        padding: 10px;
        margin: 8px 0 15px;
    }

    button {
        padding: 10px 20px;
        background: #2874f0;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    table {
        width: 100%;
        border-collapse: collapse;
    }

    th, td {
        padding: 10px;
        border: 1px solid #ddd;
    }

</style>
```

Step 10 — Test Insert

Run the application and enter:

```
Product Name: Laptop
Price: 50000
Quantity: 5
```

Click Insert Product.

Flow:

```
User
 ↓
Enter Product Details
 ↓
Click "Insert Product"
 ↓
ProductController
 ↓
_context.Products.Add()
 ↓
_context.SaveChanges()
 ↓
SQL Server
```

```
 ↓
Products Table
```

Practical 2 — CRUD Operations on Employee Table

Objective: Develop CRUD operations on an Employee table using Entity Framework Core.

Step 1 — Create Employee Model

Create Models/Employee.cs:

```
using System.ComponentModel.DataAnnotations;

namespace EFProductDemo.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }

        public string Department { get; set; }

        public decimal Salary { get; set; }
    }
}
```

Step 2 — Add Employee to DbContext

Add this property to Data/ApplicationDbContext.cs:

```
public DbSet<Employee> Employees { get; set; }
```

Complete file:

```
using EFProductDemo.Models;
using Microsoft.EntityFrameworkCore;

namespace EFProductDemo.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

Step 3 — Create Employee Table

```
Add-Migration AddEmployee
Update-Database
```

Step 4 — Create Employee Controller

Create Controllers/EmployeeController.cs:

```
using EFProductDemo.Data;
using EFProductDemo.Models;
using Microsoft.AspNetCore.Mvc;

namespace EFProductDemo.Controllers
{
    public class EmployeeController : Controller
    {
        private readonly ApplicationDbContext _context;

        public EmployeeController(
            ApplicationDbContext context)
        {
            _context = context;
        }

        // READ
        public IActionResult Index()
        {
            var employees =
                _context.Employees.ToList();

            return View(employees);
        }

        // CREATE
        [HttpPost]
        public IActionResult Create(
            Employee employee)
        {
            if (ModelState.IsValid)
            {
                _context.Employees.Add(employee);
                _context.SaveChanges();
            }

            return RedirectToAction("Index");
        }

        // UPDATE
        [HttpPost]
        public IActionResult Update(
            Employee employee)
        {
            if (ModelState.IsValid)
            {
                _context.Employees.Update(employee);
                _context.SaveChanges();
            }

            return RedirectToAction("Index");
        }

        // DELETE
        [HttpPost]
        public IActionResult Delete(int id)
        {
            var employee =
                _context.Employees.Find(id);

            if (employee != null)
            {
```

```
                _context.Employees.Remove(employee);
                _context.SaveChanges();
            }

            return RedirectToAction("Index");
        }
    }
}
```

Step 5 — Create Employee View

Create Views/Employee/Index.cshtml:

```
@model List<EFProductDemo.Models.Employee>

@{
    ViewData["Title"] = "Employee CRUD";
}

<div class="container">

    <h1>Employee Management</h1>

    <!-- CREATE -->

    <div class="box">

        <h2>Create Employee</h2>

        <form method="post"
              action="/Employee/Create">

            <input type="text"
                   name="Name"
                   placeholder="Employee Name"
                   required />

            <input type="text"
                   name="Department"
                   placeholder="Department"
                   required />

            <input type="number"
                   name="Salary"
                   placeholder="Salary"
                   step="0.01"
                   required />

            <button type="submit">
                Add Employee
            </button>

        </form>

    </div>

    <!-- READ -->

    <div class="box">

        <h2>Employee List</h2>

        @foreach (var employee in Model)
        {
```

```
            <div class="employee">

                <form method="post"
                      action="/Employee/Update">

                    <input type="hidden"
                           name="Id"
                           value="@employee.Id" />

                    <input type="text"
                           name="Name"
                           value="@employee.Name" />

                    <input type="text"
                           name="Department"
                           value="@employee.Department" />

                    <input type="number"
                           name="Salary"
                           value="@employee.Salary"
                           step="0.01" />

                    <button type="submit">
                        Update
                    </button>

                </form>

                <form method="post"
                      action="/Employee/Delete">

                    <input type="hidden"
                           name="id"
                           value="@employee.Id" />

                    <button type="submit"
                            class="delete">
                        Delete
                    </button>

                </form>

            </div>
        }

    </div>

</div>

<style>

    body {
        font-family: Arial;
        background: #f5f7fa;
    }

    .container {
        max-width: 900px;
        margin: 30px auto;
    }

    .box {
        background: white;
        padding: 25px;
```

```
        margin-bottom: 25px;
        border-radius: 10px;
        box-shadow: 0 3px 10px #ccc;
    }

    input {
        padding: 9px;
        margin: 5px;
    }

    button {
        padding: 9px 15px;
        border: none;
        border-radius: 5px;
        background: #2874f0;
        color: white;
        cursor: pointer;
    }

    .delete {
        background: #dc3545;
        margin-top: 8px;
    }

    .employee {
        padding: 15px;
        border-bottom: 1px solid #ddd;
    }

</style>
```

Step 6 — Test CRUD Operations

CREATE	**READ**	**UPDATE**	**DELETE**
Form ↓ Create() ↓ Add() ↓ SaveChanges() ↓ SQL Server	Employee Page ↓ Index() ↓ _context.Employees.ToList() ↓ Display Employees	Edit Employee ↓ Update() ↓ _context.Employees.Update() ↓ SaveChanges() ↓ SQL Server	Click Delete ↓ Delete(id) ↓ Find(id) ↓ Remove(employee) ↓ SaveChanges() ↓ SQL Server

Practical 3 — Authentication and Authorization in ASP.NET Core

Objective: Implement Authentication and Authorization in ASP.NET Core using a simple login system.

Step 1 — Install Authentication Package

In Package Manager Console run:

```
Install-Package Microsoft.AspNetCore.Authentication.Cookies
```

Step 2 — Configure Authentication

Open Program.cs and add:

```
using Microsoft.AspNetCore.Authentication.Cookies;
```

Configure cookie authentication:

```
builder.Services.AddAuthentication(
    CookieAuthenticationDefaults.AuthenticationScheme
)
.AddCookie(options =>
{
    options.LoginPath = "/Account/Login";

    options.AccessDeniedPath =
        "/Account/AccessDenied";
});
```

The middleware order should be:

```
app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();
```

Step 3 — Create Account Controller

Create Controllers/AccountController.cs:

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Mvc;

namespace EFProductDemo.Controllers
{
    public class AccountController : Controller
    {
        // LOGIN PAGE

        [HttpGet]
        public IActionResult Login()
        {
            return View();
        }

        // LOGIN PROCESS

        [HttpPost]
        public async Task<IActionResult> Login(
            string username,
            string password)
```

```
{
    // Simple demonstration credentials

    if (username == "admin" &&
        password == "1234")
    {
        var claims = new List<Claim>
        {
            new Claim(
                ClaimTypes.Name,
                username
            ),

            new Claim(
                ClaimTypes.Role,
                "Admin"
            )
        };

        var identity =
            new ClaimsIdentity(
                claims,
                CookieAuthenticationDefaults
                    .AuthenticationScheme
            );

        var principal =
            new ClaimsPrincipal(identity);

        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults
                .AuthenticationScheme,
            principal
        );

        return RedirectToAction(
            "Index",
            "Employee"
        );
    }

    ViewBag.Error =
        "Invalid username or password.";

    return View();
}

// LOGOUT

public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults
            .AuthenticationScheme
    );

    return RedirectToAction("Login");
}

// ACCESS DENIED

public IActionResult AccessDenied()
{
    return Content(
```

```
                "You are not authorized to access this page."
            );
        }
    }
}
```

Step 4 — Create Login View

Create Views/Account/Login.cshtml:

```
@{
    ViewData["Title"] = "Login";
}

<div class="login-box">

    <h1>Login</h1>

    @if (ViewBag.Error != null)
    {
        <p class="error">
            @ViewBag.Error
        </p>
    }

    <form method="post"
          action="/Account/Login">

        <label>Username</label>

        <input type="text"
               name="username"
               required />

        <label>Password</label>

        <input type="password"
               name="password"
               required />

        <button type="submit">
            Login
        </button>

    </form>

    <p>
        Demo Username: <strong>admin</strong>
    </p>

    <p>
        Demo Password: <strong>1234</strong>
    </p>

</div>

<style>

    body {
        font-family: Arial;
        background: #f5f7fa;
    }

    .login-box {
```

```
        width: 350px;
        margin: 80px auto;
        padding: 30px;
        background: white;
        border-radius: 10px;
        box-shadow: 0 3px 15px #ccc;
    }

    input {
        display: block;
        width: 95%;
        padding: 10px;
        margin: 8px 0 15px;
    }

    button {
        padding: 10px 25px;
        background: #2874f0;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    .error {
        color: red;
    }

</style>
```

Step 5 — Protect Employee Controller

Open Controllers/EmployeeController.cs and add:

```
using Microsoft.AspNetCore.Authorization;
```

Add [Authorize] above the controller:

```
[Authorize]
public class EmployeeController : Controller
{
    // CRUD methods
}
```

Step 6 — Add Logout Button

Add this to Employee/Index.cshtml:

```
<form method="get"
      action="/Account/Logout">

    <button type="submit">
        Logout
    </button>

</form>
```

Step 7 — Test Authentication

Try to open /Employee without logging in.
ASP.NET Core redirects the user to:

```
/Account/Login
```

This happens because the EmployeeController is protected by [Authorize].

Step 8 — Login

Use the demonstration credentials:

```
Username: admin
Password: 1234
```

After successful login, the application creates an authentication cookie and redirects to the Employee page.

Step 9 — Authentication vs Authorization

Authentication

Authentication asks: Who are you?

```
Username + Password
        ↓
User authenticated
```

Authorization

Authorization asks: Are you allowed to access this resource?

```
User logged in
      ↓
[Authorize]
      ↓
Employee page allowed
```

Final Combined Architecture

